

Dynamic Vertex Hierarchies for Parallel View-Dependent Progressive Meshes

Jonathan Merrin
Northeastern University
Boston, Massachusetts, USA
merrin.j@northeastern.edu

Michael Shah
Northeastern University
Boston, Massachusetts, USA
mi.shah@northeastern.edu

ABSTRACT

Triangles are one way we measure the complexity in a synthesized image. There often exists a trade-off between how realistic the synthesized image is based on the number of triangles and the overall rendering time. In real time applications, this trade-off becomes more important as we need to render detailed scenes at a fixed frame rate without sacrificing image quality. We propose a new view-dependent method that creates a more dynamic structure and is easy to parallelize. We also propose an amortized rebalancing operation to reduce long dependency lines in our data structure, prevent worst-case behavior, and in some instances improve the average case.

Our implementation is still in-progress, but our new method is provably consistent and has potential to reduce the amount of storage needed for view-dependent methods and remove more geometry at higher fidelity, along with other performance improvements.

ACM Reference Format:

Jonathan Merrin and Michael Shah. 2022. Dynamic Vertex Hierarchies for Parallel View-Dependent Progressive Meshes. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH '22 Posters)*, August 07-11, 2022. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532719.3543217>

1 INTRODUCTION

A major challenge in real-time rendering in 3D applications is balancing the number of triangles that are rendered to the screen every frame. Higher fidelity models require more triangles to approximate an object's geometry, but this additional processing on the GPU can limit the frame rate of a real-time graphics application (e.g. a game). This trade-off between detail and rendering time, as well as an increase in the complexity of 3D models, motivates the development of more efficient algorithms to simplify and refine meshes to dynamically adjust the number of triangles being rendered. Current methods of meeting this demand are hard to parallelize and often require large amounts of additional storage.

Here we demonstrate a work in progress implementation of a view-dependent progressive mesh algorithm that will take advantage of parallelism available on modern hardware.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '22 Posters, August 07-11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9361-4/22/08.

<https://doi.org/10.1145/3532719.3543217>

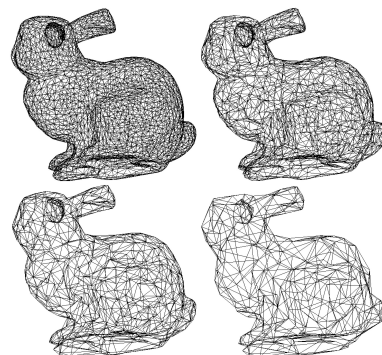


Figure 1: An example of Levels-of-Detail for the Stanford Bunny. Top-left: 4968 triangles. Top-right: 2484 triangles. Bottom-left 1242 triangles. Bottom-right: 621 triangles.

2 RELATED WORK

2.1 Global and View-Dependent Progressive Meshes

There are many established methods for simplifying a mesh [Luebke et al. 2003], but fewer that allow for dynamically re-adding detail in addition to removing. Progressive meshes (PM) outline a way to dynamically simplify and redetail arbitrary meshes using *edge collapses (ecol)* to remove geometry and *vertex splits (vsplit)* to invert an ecol. [Hoppe 1996] This allows for smooth interpolation between levels of detail (LODs), avoiding harsh popping caused by discrete LOD changes. Importantly, *vsplits* must occur in a "correct" order, or else they may introduce errors in the original mesh.

For 3D models that take up more space (e.g. terrains), a mesh can be selectively refined based on the view and what geometry is visible in the view frustum. [Hoppe 1997] and [Xia and Varshney 1996] develop a technique using precomputed vertex hierarchies to take into account the view frustum, surface orientation, and screen-space geometric error in order to perform view-dependent refinement of progressive meshes, and create a legality check for *vsplits*. It follows that all static geometry in a scene can be absorbed into a single large terrain-like mesh and simplified in this way.

[Hu et al. 2009] shows a way to parallelize View-Dependent PMs by employing an implicit legality check for *vsplit* operations. This significantly increases the efficiency of View-Dependent PMs and implicitly tracks which faces are currently included in the mesh, which significantly reduces the additional memory required to store this representation. Though this method excels at view-dependent simplification, the hierarchy that underlies it cannot be built in a view dependent way as it is constructed prior to runtime.

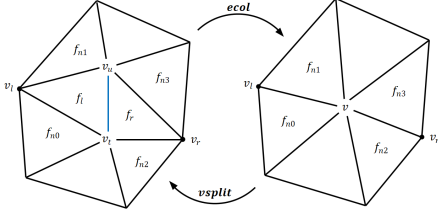


Figure 2: The inverse pair of operations *ecol* and *vsplit*.

2.2 View-Dependent Parallel Half-Edge Collapse

Hoppe points out that keeping track of triangle adjacency is difficult in parallel. This can be solved using parallel half-edge collapse [Odaker et al. 2015]. However, this work does not include a method for a parallel vertex split. Our contribution outlines a legality check for *vsplits* that is compatible with this work, allowing for view-dependent parallel progressive half-edge meshes and other benefits.

3 DYNAMIC VIEW-DEPENDENT STRUCTURES

We propose a new check for *vsplit* legality that does not rely on a pre-existing vertex hierarchy. We associate a counter with every face in the mesh. Every time an edge collapse removes a triangle, increment the counter of its three neighbors by 1. After a triangle is reintroduced to the mesh, decrement the counters of the neighbors by 1. When checking if a vertex split is legal, one must only check that the four neighbors' counters are equal. This is equivalent to the criteria outlined in [Hoppe 1997], but does not require the existence of any faces beyond the ones in the original mesh. We define an order execution of *vsplit* and *ecol* operations as *consistent* when each legal *vsplit* that reintroduces a face f to the mesh corresponds to the most recent *ecol* that removed a face adjacent f 's neighbors.

3.1 Proof of Consistency

We will now show that our legality criteria enforces a consistent order over *vsplits* and *ecols*.

Consider a mesh M that is arbitrarily simplified with a consistent order over *ecols* and *vsplits*. Let $f \in M$ be an arbitrary face in M with an associated counter $f.count$. Let v_1 be the legal *vsplit* that reintroduces a neighbor of f to M . There are only two legal actions:

- **vsplit**: Performing v_1 maintains the consistent ordering because it only increments to the next step of a consistent ordering, it does not add anything new to the order.
- **ecol**: An *ecol* that consumes a neighbor of f will increment $f.count$, invalidating v_1 and creating a new legal *vsplit*, v_2 . v_1 will only be made legal again once v_2 is performed and $f.count$ is decremented. Since v_2 corresponds to the newly most recent *ecol* that consumed a neighbor of f , this new ordering is consistent.

The rest follows because the initial mesh has no legal *vsplits*, and so has a consistent ordering by default. ■

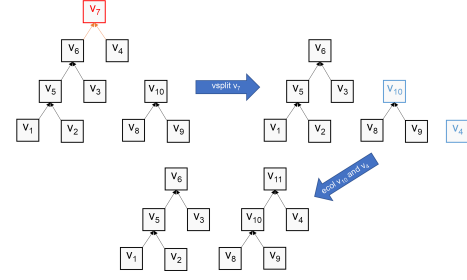


Figure 3: Our rebalancing operation reduces the longest dependency line by 1. Note in this example there must be an edge between v_4 and v_{10} .

Our method allows us to generate the vertex hierarchy at run-time because it does not rely on any features of an existing vertex hierarchy. This requires keeping track of face adjacency, which can be done with half-edges, and allows the data structure to be dynamically constructed in a view-dependent way rather than using a static, precomputed structure.

Dynamic structures could increase the chance of worst case behavior in the form of long dependency lines. For this we propose an amortized rebalancing operation (Figure 2). Conveniently, our counter also serves to keep track of how many *vsplits* we would need to get to the original faces in the mesh. We mark a face with a dependency line that is more than two greater than any of its neighbors as a candidate for rebalancing. Then in the rebalancing step, we can perform a *vsplit* to reintroduce two faces, then merge them with two of the *vsplit*'s neighbors, as seen in Figure 3. With any leftover time in a frame, we can perform this rebalancing operation to reduce the likelihood of this worst case scenario.

4 CONCLUSIONS AND FUTURE WORK

Upon completion of the implementation and benchmarking of the above work, we would like to analyze the effect of including the amortized rebalancing operation against our baseline. Afterwards we would like to explore other applications of our system, including reducing meshes used for shadow-casting and collision detection.

REFERENCES

- Hugues Hoppe. 1996. Progressive meshes. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*. 99–108.
- Hugues Hoppe. 1997. View-dependent refinement of progressive meshes. In *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*. 189–198.
- Liang Hu, Pedro V. Sandler, and Hugues Hoppe. 2009. Parallel View-Dependent Refinement of Progressive Meshes. In *Symposium on Interactive 3D Graphics and Games (I3D)*. 169–176.
- David Luebke, Martin Reddy, Jonathan D Cohen, Amitabh Varshney, Benjamin Watson, and Robert Huebner. 2003. *Level of detail for 3D graphics*. Morgan Kaufmann.
- Thomas Odaker, Dieter Kranzmueller, and Jens Volkert. 2015. View-dependent simplification using parallel half edge collapses. In *SCG 2015 Conference on Computer Graphics, Visualization and Computer Vision*.
- Julie C Xia and Amitabh Varshney. 1996. Dynamic view-dependent simplification for polygonal models. In *Proceedings of Seventh Annual IEEE Visualization '96*. IEEE, 327–334.